



Programmierung interaktiver Systeme WS2022/23

Programmierprojekt

BATTLESHIP (Kartenspiel)

SEYIT AHMET DUMAN – 5401532



Brief information about Battleship-Game

- Battleship is a card game played in twos.
- The aim of the game is to sink all 5 enemy ships.
- Each player has 12 grid cards consisting of 5 ship cards and 7 miss cards
- 23 battle cards consisting of 4 rescue cards and 19 missile cards.
- Grid cards are revealed by missile cards and if a ship is revealed, it is sunk with it.
- Ships and pin cards have different strengths and powers.
- Rescue cards are to be played on your own cards. These are repair and shield cards.

Brief information about Battleship-CardGame Project

- In the project, this is realized with two Battleship Game instances.
- One side is implemented as server and the other side as client.
- Threads in View are responsible for network communication and card/information updates.
- Updated data and cards are exchanged through them and stored/set in the model via controllers.
- Server is always configured as game beginner and as 'red' team. Client, on the other hand, is 'blue'.
- Thanks to the 'turn' variable, the two players are not allowed to play at the same time.
- During the game, the players are informed by the info boxes.
- When the card is played, it is automatically saved to the deposit cards and a new card is saved from the battle cards to hand.
- In the beginning, each player automatically has 5 cards in hand.
- If all cards from battle cards should be used, the deposit cards are shuffled again and placed on battle cards.
- If a player sinks all opponent's ships, the game ends, this player also informs opponents about it, so that they also end the game.

General rules for the game

- Missile cards are to be played on your opponent's card.
- Rescue cards (shield and repair), on the other hand, are on their own cards.
- Rescue cards can only be played on your own ship cards that have already been revealed by the opponent.
- Repair cards can only be played on your own ship cards that have already been hit.
- Repair card puts +1 power to the ship. And Shield card puts +2 power to the ship.
- Both red and white pin cards can be used to uncover. But if the opponent's card is a submarine, it will count as nothing and the player will lose his card. (This is deleted from the game, not sent to the stock cards.)
- U-Boot can only be hit with white pin cards. If the missile is red, it will count as nothing and the player will lose his card. This is deleted from the game.
- When one side's battleship is revealed by enemy, each attack of that side has +1 power on enemy ship. This applies until this ship (battleship) is sunk.

General information about GUI

- Note: Since the size of the game is 1000*1000, it should be played with either a larger external screen or with 100% screen scaling.
- Players are informed from the start always with the info boxes.
- Server is created and title screen is loaded from the game. There player can change the player name.
- The game is loaded with the start button. Then, the player waits for the client(opponent) to connect.
- When this occurs, players will trade GameCards with each other, and these will be stored in Model via controllers.
- When it is one side's turn, the other side is not allowed to play.
- The info boxes on the right corner tell you whose turn it is.
- Information about the shiptype in GUI
 - s - stands for Submarine
 - d - stands for Destroyer
 - b - stands for Battleship
 - c - stands for Cruiser
 - a – stands for Aircraft Carrier

Instructions for GUI

- **Space:** At the beginning of the game, after the server and client are connected, the players will swap their grid cards with each other using the **space key**.
- **Keys(1 to 5):** To play on the GUI, it should be first HandCard selected with the **keys (1 to 5)**
- **Mouse Click:** Then click on the opponent's card or own card.
- With the info box on the right results or warning/error messages can be seen.
-

General information about JShell file

- The code file for JShell was put in separately.
- The JShell file can be opened and played directly in JShell.
- In JShell, a separate move method is used to print the card information after each move.
- This method also calls the main move method but additionally returns the information.

What means Symbols in Jshell

- [W1] for white missile with power of 1
- [R2] for red missile with power of 2
- [R4] for red missile with power of 4
- [R1] for red missile with power of 1
- [+] for repair card
- (S) for shield card
- [X] for missed
- [/] for sunk ship
- [A5] for aircraft carrier with current force
- [S3] for submarine with current force

- [B4]] for battleship with current force
- [C3]] for cruiser with current force
- [D2]] for destroyer with current force

```

C:\Windows\System32\cmd.exe X + v
Microsoft Windows [Version 10.0.22621.1105]
(c) Microsoft Corporation. Alle Rechte vorbehalten.

C:\Users\duman\Desktop\THM_MNI_II\2. Semester\Pis\Battleship_JShell>jshell
| Willkommen bei JShell - Version 19.0.2
| Geben Sie für eine Einführung Folgendes ein: /help intro

jshell> /open BattleshipGame.java

jshell> BattleshipGame bsR = new BattleshipGame("red")
bsR ==>
GameCards of red
[] [] [] []
[] [] [] []
[] [] [] []

Han ... f red
[W1][R2][W1][W1][R1]

jshell> BattleshipGame bsB = new BattleshipGame("blue")
bsB ==>
GameCards of blue
[] [] [] []
[] [] [] []
[] [] [] []

Ha ... blue
[R2][W1][R1][W1][R1]

jshell>

```

Instructions for JShell

- Open the JShell file in jshell.
/open BattleshipGame.java
- Create two BattleshipGame instances.
BattleshipGame bsR = new BattleshipGame("red")
BattleshipGame bsB = new BattleshipGame("blue")
- Playing with "moveJ(Cards handCard, Cards gameCard, BattleshipGame enemy)"
moveJ(bsR.getHandCards().get(0), bsB.getGameCards().get(5), bsB)

```

jshell>
bsR.moveJ(bsR.getHandCards().get(1), bsB.getGameCards().get(5), bsB)
Enemy-Ship is discovered!!! Type: b
Defence: 4
Enemy-Ship was attacked!!! Type: b
Defence: 2

GameCards of red
[] [] [] []
[] [] [] []
[] [] [] []

HandCards of red
[W1][W1][W1][R1](S)

GameCards of blue
[] [] [] []
[] [B2] [] []
[] [] [] []

HandCards of blue
[R2][W1][R1][W1][R1]

jshell>

```

```

jshell> bsB.moveJ(bsB.getHandCards().get(2), bsR.getGameCards().get(9), bsR)
Enemy-Ship is discovered!!! Type: b
Defence: 4
Enemy-Ship was attacked!!! Type: b
Defence: 2

GameCards of blue
[] [] [] []
[] [B2] [] []
[] [] [] []

HandCards of blue
[R2][W1][W1][R1][W1]

GameCards of red
[] [] [] []
[] [] [] []
[] [B2] [] []

HandCards of red
[W1][W1][W1][R1](S)

jshell>

```

```

jshell> bsR.moveJ(bsR.getHandCards().get(3), bsB.getGameCards().get(5), bsB)
Enemy-Ship was attacked!!! Type: b
Defence: 0
Enemy-Ship was sunk!!! Type: b
Defence: 0

GameCards of red
[] [] [] []
[] [] [] []
[] [B2] [] []

HandCards of red
[W1][W1][W1](S)[R1]

GameCards of blue
[] [] [] []
[] [/] [] []
[] [] [] []

HandCards of blue
[R2][W1][W1][R1][W1]

jshell>

```

The language and external libraries

- JDK 17.0.4.1 is used for the project and
 - <https://www.oracle.com/java/technologies/javase/jdk17-archive-downloads.html>
- Core and controlP5 from Processing are used as external libraries.
 - <https://processing.org/reference/libraries/>
 - <https://sojamo.de/libraries/controlP5/>

Source codes for images and fonts

- 1) **SgtJhonO / Demo-non Commercial**
<https://www.fontspace.com/sgt-jhon-o-font-f48444>
- 2) **BravoDestroyer / Demo-non Commercial**
<https://www.fontspace.com/bravo-destroyer-font-f29241>
- 3) **Gunmetal / Freeware**
<https://www.fontspace.com/gunmetal-font-f5085>
- 4) **Maldini / Demo-non Commercial**
<https://www.fontspace.com/maldini-font-f39870>

Images were downloaded from freepik with premium license and designed by the author.
Freepik premium license does not require attribution.

<https://de.freepik.com/>
https://support.freepik.com/s/article/What-are-Freepik-Premium-licenses?language=en_US

Enclosurement

Screenshots of Game Screen.

